# routing tables

| IP addresses | gateway | iface |
|---|---|---|
| 2001:0db8:40:f000::/44 | --- | int1 |
| 2001:0db8:40:e000::/44 | 2001:0db8:40:f000::2 | int1 |
| 2001:0db8:40:d000::/44 | --- | int3 |
| 3fff:1000:19::/48 | --- | ext1 |
| … | … | … |
| default | fe80::17 | ext2 |

| IP addresses | gateway | iface |
|---|---|---|
| 192.0.2.0/25 | --- | int1 |
| 192.0.2.128/26 | 192.0.2.1 | int1 |
| 192.0.2.192/26 | 192.0.2.2 | int1 |
| 198.51.100.0/25 | 192.0.2.1 | int1 |
| 198.51.100.128/25 | --- | int2 |
| … | … | … |
| default | 203.0.113.1 | ext |

# filling routing tables

easy part: what networks are you directly connected to

- that range of IP addresses, that interface

harder part: other routers on connected router

need to learn:

- addresses of other router
- which networks can be reached through them directly or indirectly

need to choose between multiple ways of reaching networks

# problems when forwarding

no entry in routing table

no entry in neighbor table

(after attempting ARP, or neighbor discovery)

packet too big for next network

there's an infinite loop in the route

# problems when forwarding

no entry in routing table

no entry in neighbor table

(after attempting ARP, or neighbor discovery)

packet too big for next network

there's an infinite loop in the route

# destination host unreachable

```
$ ping 128.143.67.254
PING 128.143.67.254 (128.143.67.254) 56(84) bytes of data.
From 128.143.63.1 icmp_seq=1 Destination Host Unreachable
From 128.143.63.1 icmp_seq=6 Destination Host Unreachable
^C
--- 128.143.67.254 ping statistics ---
10 packets transmitted, 0 received, +2 errors, 100% packet los
pipe 4
```

—

```
$ ping6 2606:8e80:7007:ef1a::1
PING 2606:8e80:7007:ef1a::1(2606:8e80:7007:ef1a::1)
From 2606:8e80:7007:ef1a:cf1f:3948:b5c1:a522 icmp_s
....
```

# ICMPv6 destination unreachable messages

IPv6 header with ICMP as next protocol

1 byte type = 1 (destination unreachable)

1 byte code =

- examples: address unreachable, administritatively prohibited

most of contents of message causing problem

- only most to avoid exceeding max packet size
- should let OS figure out which socket to send error to

# generating destination unreachable

by routers: reached correct network, machine not there

by routers: no route to network at all

by routers: administrator rule prohibits forwarding

by destination host: no program listening to that 'port'

...

different code values for all cases

machine can also choose to send nothing back

# ICMPv4 destination unrachable

basically same format as ICMPv6, but...

different type/code integer values

only IPv4 header + 64 bytes of original packet included

# problems when forwarding

no entry in routing table

no entry in neighbor table

(after attempting ARP, or neighbor discovery)

packet too big for next network

there's an infinite loop in the route

# fragmentation

max frame data size on my local network = 1500 bytes, but...

```
$ ping6 fe80::da07:b6ff:fed9:ae50 -s 4000
PING fe80::da07:b6ff:fed9:ae50 (fe80::da07:b6ff:fed9:ae50) 40
4008 bytes from fe80::da07:b6ff:fed9:ae50%eno1: icmp_seq=1 tt
4008 bytes from fe80::da07:b6ff:fed9:ae50%eno1: icmp_seq=2 tt
4008 bytes from fe80::da07:b6ff:fed9:ae50%eno1: icmp_seq=3 tt
...
$ ping -s 4000 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 4000(4028) bytes of data.
4008 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.891 ms
4008 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.806 ms
4008 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.748 ms
```

# fragmentation

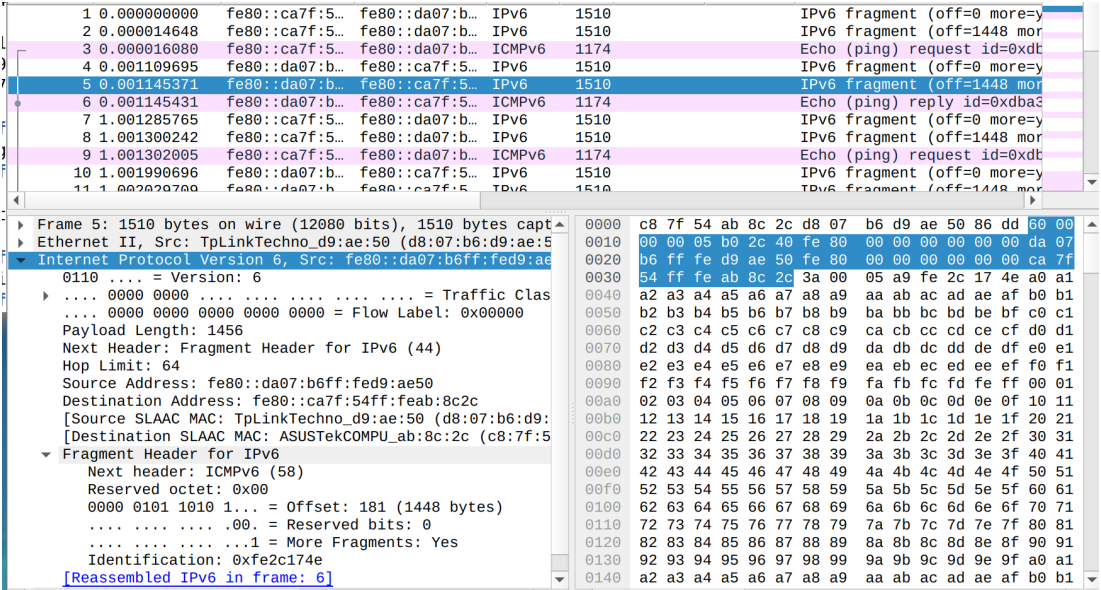
original sender or router splits packet into multiple

each part called a *fragment*

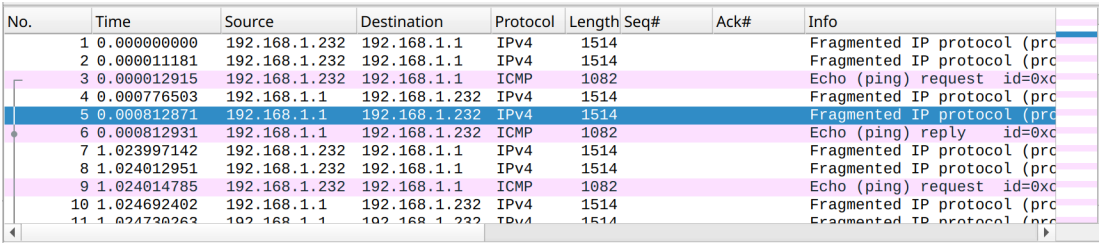
stored temporarily and "reassembled" at receiver

Linux defaults:
max 64 packet gap between fragments per source IP
30 second time limit before discaded
3-4MB buffer of packets

# IPv6 fragments



# IPv4 fragments



| No. | Time | Source | Destination | Protocol | Length | Seq# | Ack# | Info |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 192.168.1.232 | 192.168.1.1 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 2 | 0.000011181 | 192.168.1.232 | 192.168.1.1 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 3 | 0.000012915 | 192.168.1.232 | 192.168.1.1 | ICMP | 1082 | | | Echo (ping) request id=0xc |
| 4 | 0.000776503 | 192.168.1.1 | 192.168.1.232 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 5 | 0.000812871 | 192.168.1.1 | 192.168.1.232 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 6 | 0.000812931 | 192.168.1.1 | 192.168.1.232 | ICMP | 1082 | | | Echo (ping) reply id=0xc |
| 7 | 1.023997142 | 192.168.1.232 | 192.168.1.1 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 8 | 1.024012951 | 192.168.1.232 | 192.168.1.1 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 9 | 1.024014785 | 192.168.1.232 | 192.168.1.1 | ICMP | 1082 | | | Echo (ping) request id=0xc |
| 10 | 1.024692402 | 192.168.1.1 | 192.168.1.232 | IPv4 | 1514 | | | Fragmented IP protocol (prc |
| 11 | 1.024730263 | 192.168.1.1 | 192.168.1.232 | IPv4 | 1514 | | | Fragmented IP protocol (prc |

```
▸ Frame 5: 1514 bytes on wire (12112 bits), 1514 bytes captur
▸ Ethernet II, Src: TpLinkTechno_d9:ae:50 (d8:07:b6:d9:ae:50)
▾ Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not
    Total Length: 1500
    Identification: 0xaa6b (43627)
  ▸ 001. .... = Flags: 0x1, More fragments
    ...0 0000 1011 1001 = Fragment Offset: 1480
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x25c3 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.1.1
    Destination Address: 192.168.1.232
    [Reassembled IPv4 in frame: 6]
▸ Data (1480 bytes)
  [Community ID: 1:Ac2BWwekehTAnPxIuiD9sNt+ItE=]
```

```
0000  c8 7f 54 ab 8c 2c d8 07  b6 d9 ae 50 08 00 45 00
0010  05 dc aa 6b 20 b9 40 01  25 c3 c0 a8 01 01 c0 a8
0020  01 e8 c0 c1 c2 c3 c4 c5  c6 c7 c8 c9 ca cb cc cd
0030  ce cf d0 d1 d2 d3 d4 d5  d6 d7 d8 d9 da db dc dd
0040  de df e0 e1 e2 e3 e4 e5  e6 e7 e8 e9 ea eb ec ed
0050  ee ef f0 f1 f2 f3 f4 f5  f6 f7 f8 f9 fa fb fc fd
0060  fe ff 00 01 02 03 04 05  06 07 08 09 0a 0b 0c 0d
0070  0e 0f 10 11 12 13 14 15  16 17 18 19 1a 1b 1c 1d
0080  1e 1f 20 21 22 23 24 25  26 27 28 29 2a 2b 2c 2d
0090  2e 2f 30 31 32 33 34 35  36 37 38 39 3a 3b 3c 3d
00a0  3e 3f 40 41 42 43 44 45  46 47 48 49 4a 4b 4c 4d
00b0  4e 4f 50 51 52 53 54 55  56 57 58 59 5a 5b 5c 5d
00c0  5e 5f 60 61 62 63 64 65  66 67 68 69 6a 6b 6c 6d
00d0  6e 6f 70 71 72 73 74 75  76 77 78 79 7a 7b 7c 7d
00e0  7e 7f 80 81 82 83 84 85  86 87 88 89 8a 8b 8c 8d
00f0  8e 8f 90 91 92 93 94 95  96 97 98 99 9a 9b 9c 9d
0100  9e 9f a0 a1 a2 a3 a4 a5  a6 a7 a8 a9 aa ab ac ad
0110  ae af b0 b1 b2 b3 b4 b5  b6 b7 b8 b9 ba bb bc bd
0120  be bf c0 c1 c2 c3 c4 c5  c6 c7 c8 c9 ca cb cc cd
```

# varying frame size support

also called *maximum transmission unit* (MTU)

typical Ethernet, Wifi — 1500 bytes

Ethernet with "jumbo frames" – 65535 bytes

IPsec ESP VPN over 1500-byte MTU network – ∼1400–1440 bytes

VPN — simulated network link over other network links

# routers making fragments

option in IPv4 to handle frame size mismatch, but not great:

extra data sent over network (especially if just over max size)

- extra copies of main headers on each fragment

extra work at receiver to reconstruct fragments

lose whole packet if one fragment is lost

- but other routers likely to still waste time forwarding all other fragments

# avoiding fragmentation

IPv4 — DF (don't fragment) flag in packets

- if set, routers not allowed to fragment packet

IPv6 — routers never fragment packets

- any fragments made at source machine only

# avoiding fragmentation

IPv4 — DF (don't fragment) flag in packets

- if set, routers not allowed to fragment packet

IPv6 — routers never fragment packets

- any fragments made at source machine only

when set — ICMP error

- ICMPv6: Packet Too Big
- ICMPv4: destination unreachable + reason code of fragmentation needed
- (hopefully, bad networks might drop packet instead)

ICMPv6 error tells you maximum supported size

- (by first link that got packet rejected — might be more constraining link later)

# exercise: fragmentation perf

assume:

- Ethernet header/trailer: 26 bytes
- IPv4 header: 20 bytes + 0 bytes of options
- TCP header: 20 bytes + 16 bytes of options

suppose local network supports 65535-byte ethernet payloads

and remote network suports 1500-byte ethernet payloads

and fragmentation happens

exericses:

- lowest overhead TCP segment size?
- overhead for 64000-byte TCP segments?
- highest overhead TCP segment size?

# problems when forwarding

no entry in routing table

no entry in neighbor table

(after attempting ARP, or neighbor discovery)

packet too big for next network

there's an infinite loop in the route

# time-to-live (v4) / hop limit (v6)

stored in IP header

when forwarding packet, router will:

subtract one from TTL / hop limit

and recompute checksum accordingly

if TTL/hop limit = 0, drop packet

usually send back ICMP “Time Exceeded” error

# traceroute

ICMP Time Exceeded messages come from router

$\rightarrow$ tells you which routers are involved

# traceroute

ICMP Time Exceeded messages come from router

$\rightarrow$ tells you which routers are involved

`traceroute` command: deliberately packets with low TTL/hop limit

print out what time exceeded messages we get back

typically sent with TTL/hop limit = 255 so it doesn't get lost

('backwards' path might be longer than forwards one)

# traceroute example

```
traceroute to ripe.net (193.0.11.51), 30 hops max, 60 byte packets
 1  128.143.63.1 (128.143.63.1)  6.367 ms  8.562 ms  8.577 ms
 2  cr01-gil-ae15-00.net.virginia.edu (128.143.221.17)  0.370 ms  0.334 ms  0.349 ms
 3  * * *
 4  br01-udc-et-1-2-0.net.virginia.edu (128.143.236.5)  0.502 ms  0.468 ms  0.488 ms
 5  i2-vt.net.virginia.edu (192.35.48.34)  3.374 ms  3.448 ms  3.413 ms
 6  192.122.175.15 (192.122.175.15)  5.715 ms  5.628 ms  5.590 ms
 7  fourhundredge-0-0-0-17.4079.core1.ashb.net.internet2.edu (163.253.1.8)  29.163 ms
    fourhundredge-0-0-0-16.4079.core1.ashb.net.internet2.edu (163.253.1.2)  28.880 ms
    fourhundredge-0-0-0-17.4079.core1.ashb.net.internet2.edu (163.253.1.8)  28.876 ms
 8  fourhundredge-0-0-0-1.4079.core1.clev.net.internet2.edu (163.253.1.123)  29.568 ms
    28.667 ms  28.666 ms
 9  fourhundredge-0-0-0-0.4079.core2.newy32aoa.net.internet2.edu (163.253.1.239)  29.608 ms
    29.476 ms  29.400 ms
10  fourhundredge-0-0-0-19.4079.core1.newy32aoa.net.internet2.edu (163.253.1.40)  28.958 ms
    28.999 ms
    fourhundredge-0-0-0-21.4079.core1.newy32aoa.net.internet2.edu (163.253.1.44)  29.280 ms
11  e1-3-2-502.asd001b-jnx-06.surf.net (145.145.166.18)  115.822 ms  115.823 ms  115.744 ms
12  lo0-2.asd001b-jnx-01-surfinternet.surf.net (145.145.128.4)  115.988 ms  115.932 ms  115.
13  gw.amsix.telrtr.ripe.net (80.249.208.71)  121.956 ms  121.968 ms  121.844 ms
14  * * *
15  * * *
```

# traceroute sent

| No. | Time | Source | Destination | TTL | Protocol | Length | Info |
|---|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 128.143.71.27 | 193.0.11.51 | 1 | UDP | 74 | 54510 → 33434 Len=32 |
| 2 | 0.000042000 | 128.143.71.27 | 193.0.11.51 | 1 | UDP | 74 | 56464 → 33435 Len=32 |
| 3 | 0.000078065 | 128.143.71.27 | 193.0.11.51 | 1 | UDP | 74 | 36104 → 33436 Len=32 |
| 4 | 0.000115226 | 128.143.71.27 | 193.0.11.51 | 2 | UDP | 74 | 34004 → 33437 Len=32 |
| 5 | 0.000151405 | 128.143.71.27 | 193.0.11.51 | 2 | UDP | 74 | 57973 → 33438 Len=32 |
| 6 | 0.000186502 | 128.143.71.27 | 193.0.11.51 | 2 | UDP | 74 | 50866 → 33439 Len=32 |
| 7 | 0.000222625 | 128.143.71.27 | 193.0.11.51 | 3 | UDP | 74 | 48263 → 33440 Len=32 |
| 8 | 0.000257734 | 128.143.71.27 | 193.0.11.51 | 3 | UDP | 74 | 60098 → 33441 Len=32 |
| 9 | 0.000292642 | 128.143.71.27 | 193.0.11.51 | 3 | UDP | 74 | 58655 → 33442 Len=32 |
| 10 | 0.000327537 | 128.143.71.27 | 193.0.11.51 | 4 | UDP | 74 | 40741 → 33443 Len=32 |
| 11 | 0.000362342 | 128.143.71.27 | 193.0.11.51 | 4 | UDP | 74 | 48193 → 33444 Len=32 |
| 12 | 0.000397460 | 128.143.71.27 | 193.0.11.51 | 4 | UDP | 74 | 60985 → 33445 Len=32 |
| 13 | 0.000433117 | 128.143.71.27 | 193.0.11.51 | 5 | UDP | 74 | 38126 → 33446 Len=32 |
| 14 | 0.000468223 | 128.143.71.27 | 193.0.11.51 | 5 | UDP | 74 | 38788 → 33447 Len=32 |

# traceroute received

```
    30 0.003908850   192.35.48.34      128.143.71.27     251,1 ICMP     110 Time-to-live exceeded
    31 0.003908954   192.35.48.34      128.143.71.27     251,1 ICMP     110 Time-to-live exceeded
    35 0.006246570   192.122.175.15    128.143.71.27     250,1 ICMP     110 Time-to-live exceeded
    36 0.006246708   192.122.175.15    128.143.71.27     250,1 ICMP     110 Time-to-live exceeded
    37 0.006246784   192.122.175.15    128.143.71.27     250,1 ICMP     110 Time-to-live exceeded
    38 0.006346847   128.143.63.1      128.143.71.27      64,1 ICMP      70 Time-to-live exceeded
    43 0.008594648   128.143.63.1      128.143.71.27      64,1 ICMP      70 Time-to-live exceeded
    44 0.008647695   128.143.63.1      128.143.71.27      64,1 ICMP      70 Time-to-live exceeded
    50 0.029854913   163.253.1.8       128.143.71.27     243,1 ICMP     186 Time-to-live exceeded
    51 0.029855048   163.253.1.2       128.143.71.27     243,1 ICMP     186 Time-to-live exceeded
    52 0.029904130   163.253.1.8       128.143.71.27     243,1 ICMP     186 Time-to-live exceeded
    53 0.030634797   163.253.1.123     128.143.71.27     244,2 ICMP     186 Time-to-live exceeded
    54 0.031475863   128.143.236.90    128.143.71.27     253,1 ICMP      70 Time-to-live exceeded
    55 0.031476027   128.143.236.90    128.143.71.27     253,1 ICMP      70 Time-to-live exceeded
```

# aside: multiple paths

only showing *forward* path

- routing in reverse direction is often different

sometimes multiple forward paths

- way we've shown routing table so far does not allow this

# constructing routing/neighbor tables

interesting task: how to fill tables

two general strategies:

routers/switches learn from neighbors

> "distributed"

information gathered on single controller machine which configures routers/switches

> "centralized"

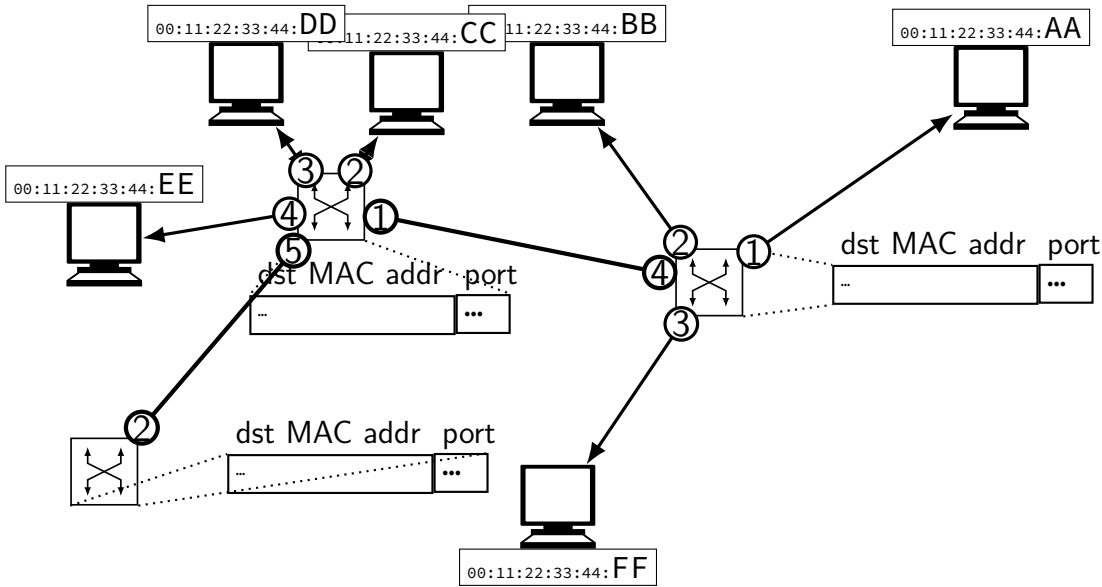
# basic flooding

idea: broadcast message to whole network

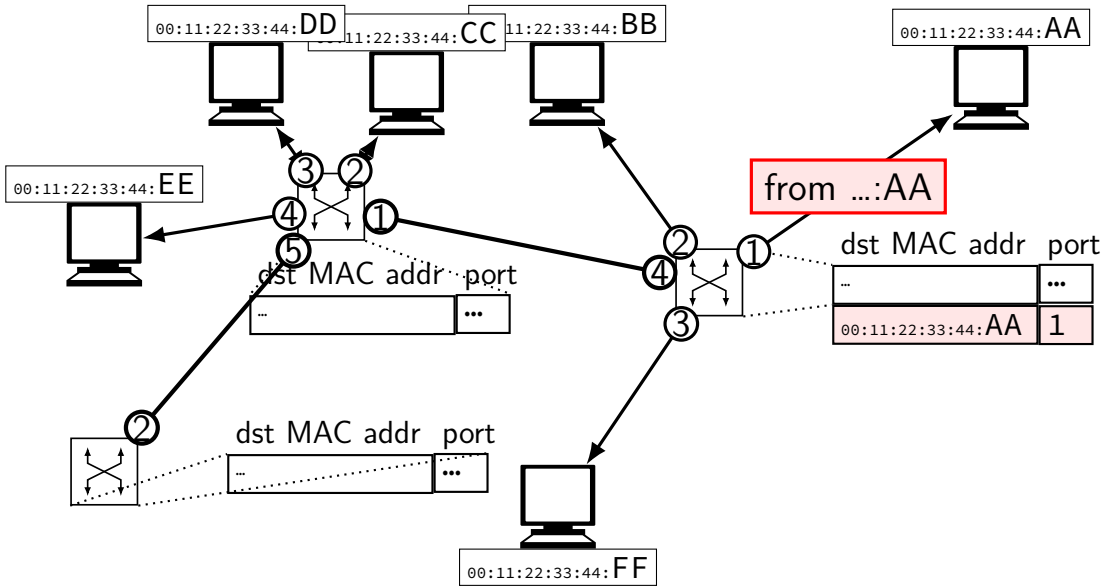
where message comes from = way to send back

used this idea in MAC learning

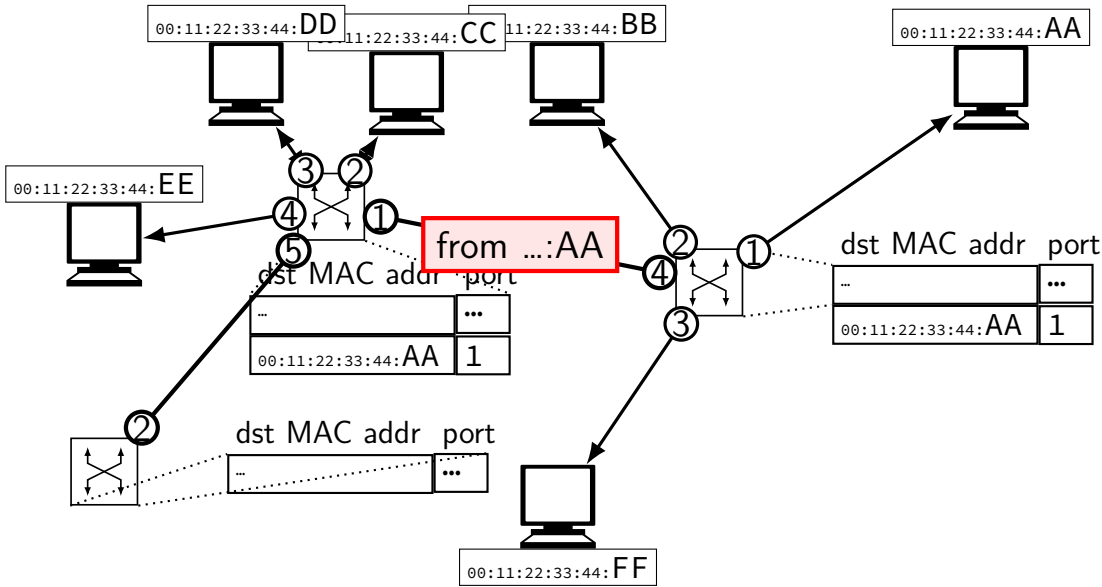
# flooding one entry



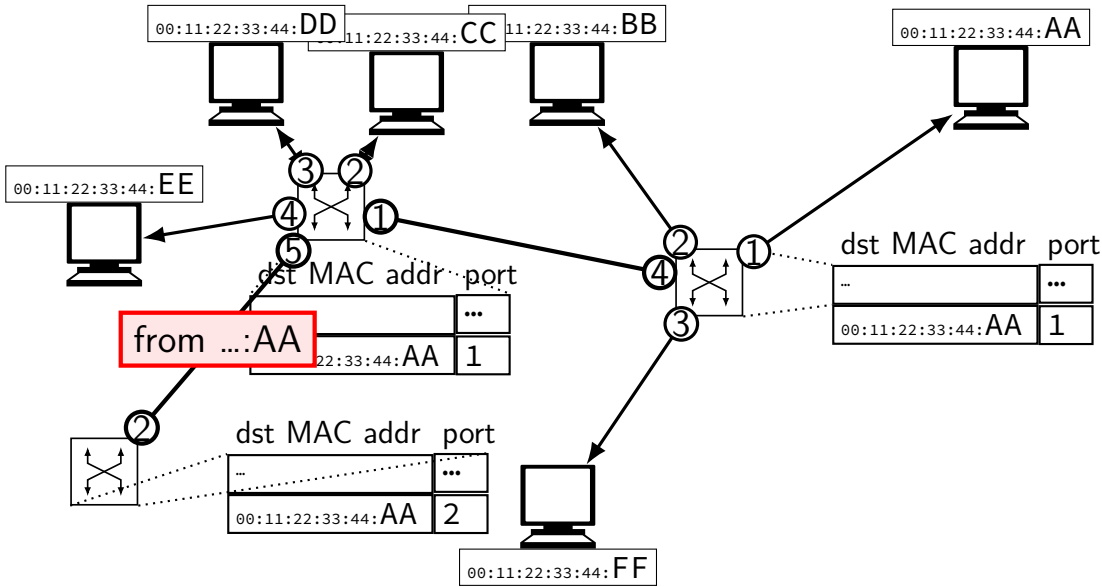
# flooding one entry



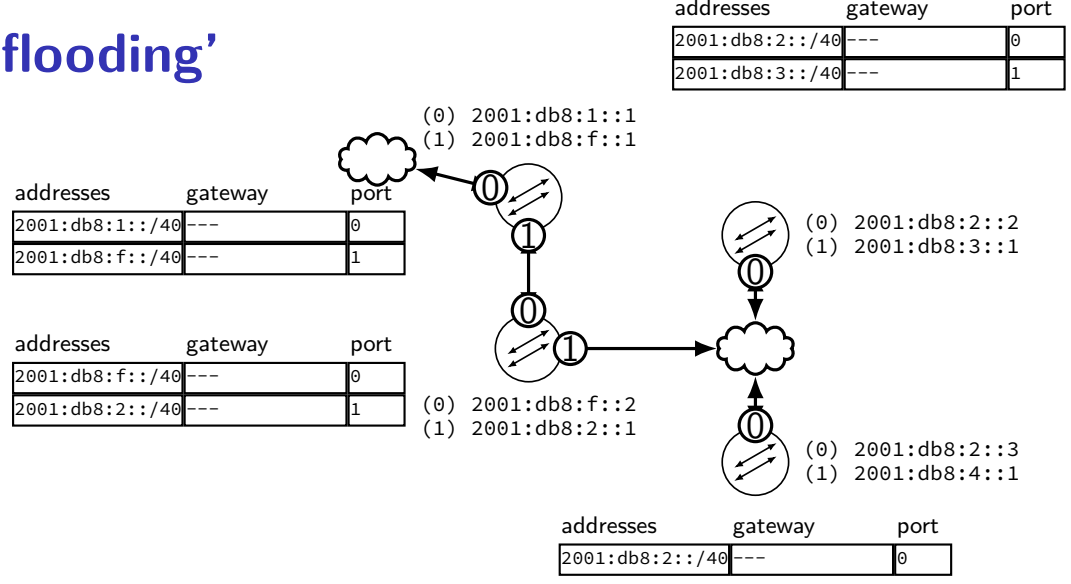
# flooding one entry



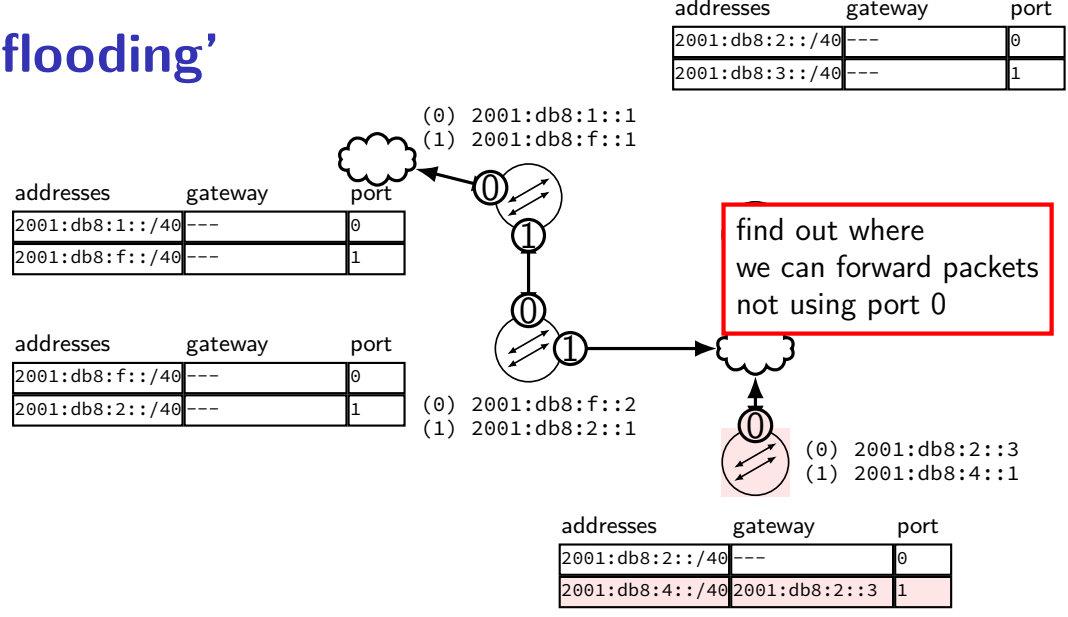
# flooding one entry



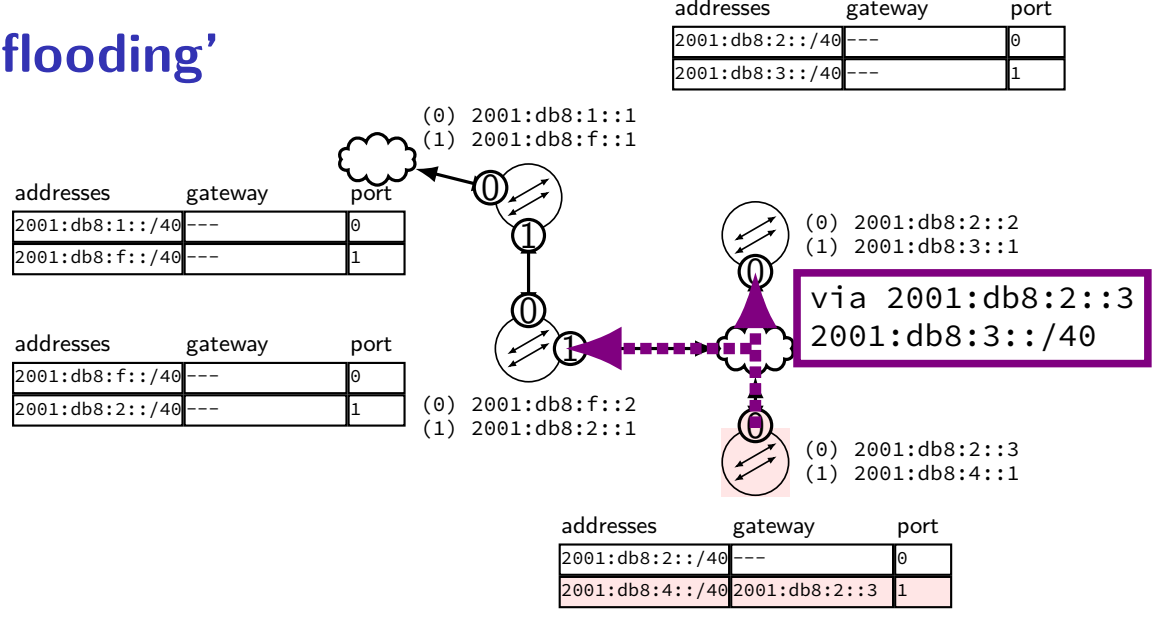
# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
2001:db8:1::/40 --- 0
2001:db8:f::/40 --- 1
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
(0) 2001:db8:f::2
(1) 2001:db8:2::1
(0) 2001:db8:2::2
(1) 2001:db8:3::1
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0

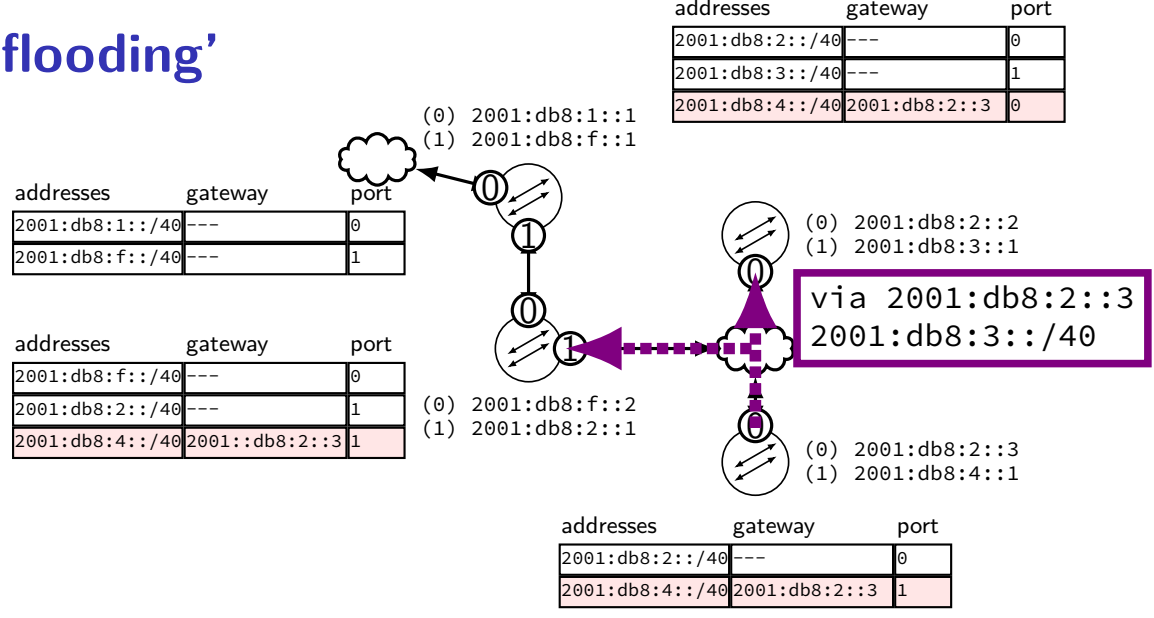
# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
2001:db8:1::/40 --- 0
2001:db8:f::/40 --- 1
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
(0) 2001:db8:f::2
(1) 2001:db8:2::1
find out where
we can forward packets
not using port 0
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:4::/40 2001:db8:2::3 1

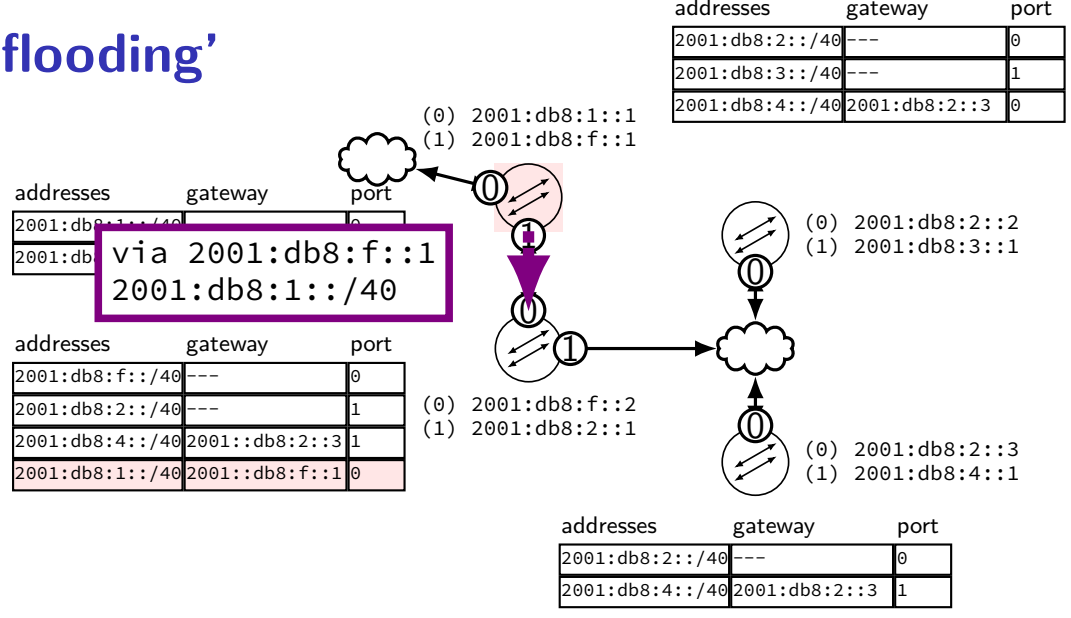
# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
2001:db8:1::/40 --- 0
2001:db8:f::/40 --- 1
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
(0) 2001:db8:f::2
(1) 2001:db8:2::1
(0) 2001:db8:2::2
(1) 2001:db8:3::1
via 2001:db8:2::3
2001:db8:3::/40
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:4::/40 2001:db8:2::3 1

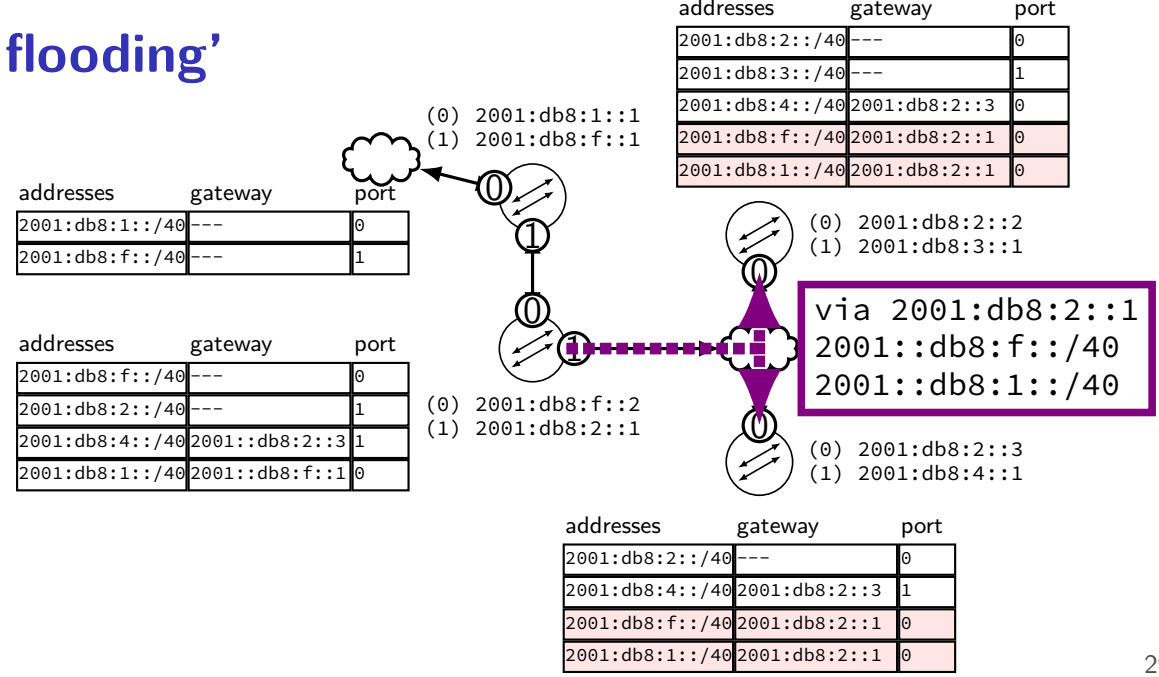
# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
2001:db8:4::/40 2001:db8:2::3 0
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
2001:db8:1::/40 --- 0
2001:db8:f::/40 --- 1
(0) 2001:db8:2::2
(1) 2001:db8:3::1
via 2001:db8:2::3
2001:db8:3::/40
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
2001:db8:4::/40 2001::db8:2::3 1
(0) 2001:db8:f::2
(1) 2001:db8:2::1
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:4::/40 2001:db8:2::3 1

# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
2001:db8:4::/40 2001:db8:2::3 0
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
via 2001:db8:f::1
2001:db8:1::/40
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
2001:db8:4::/40 2001::db8:2::3 1
2001:db8:1::/40 2001::db8:f::1 0
(0) 2001:db8:f::2
(1) 2001:db8:2::1
(0) 2001:db8:2::2
(1) 2001:db8:3::1
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:4::/40 2001:db8:2::3 1

# ‘flooding’


addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:3::/40 --- 1
2001:db8:4::/40 2001:db8:2::3 0
2001:db8:f::/40 2001:db8:2::1 0
2001:db8:1::/40 2001:db8:2::1 0
(0) 2001:db8:1::1
(1) 2001:db8:f::1
addresses gateway port
2001:db8:1::/40 --- 0
2001:db8:f::/40 --- 1
(0) 2001:db8:2::2
(1) 2001:db8:3::1
via 2001:db8:2::1
2001::db8:f::/40
2001::db8:1::/40
addresses gateway port
2001:db8:f::/40 --- 0
2001:db8:2::/40 --- 1
2001:db8:4::/40 2001::db8:2::3 1
2001:db8:1::/40 2001::db8:f::1 0
(0) 2001:db8:f::2
(1) 2001:db8:2::1
(0) 2001:db8:2::3
(1) 2001:db8:4::1
addresses gateway port
2001:db8:2::/40 --- 0
2001:db8:4::/40 2001:db8:2::3 1
2001:db8:f::/40 2001:db8:2::1 0
2001:db8:1::/40 2001:db8:2::1 0

# eventual convergence

'flooding' algorithm:

periodically send on each network:

> list of routes you have that don't double-back to same network

when receiving routes sent on network:

> add routing table entry for each route

# eventual convergence

'flooding' algorithm:

periodically send on each network:

> list of routes you have that don't double-back to same network

when receiving routes sent on network:

> add routing table entry for each route

not handled: multiple paths?

# only one path?

only one path on network means:

if a link fails, bad news

network forms a tree

# routing like this?

for IP routing, generally want to have multiple paths

...but this is basically how MAC learning works

but it requires a network that is a tree

what if we don't start with one?

# spanning tree

given a general network, only activate subset of links

...such that network is tree

that is only one path between each node

allows us to do flooding strategy

makes simple MAC learning/broadcast just work

# centralized spanning tree?

one algorithm you might learn in DSA2:

mark one node called *the root* as 'in the tree'

repeatedly:

> add the 'first' link that goes to a node not in the tree
> mark newly connected node as 'in the tree'

result = spanning tree

# centralized spanning tree?

one algorithm you might learn in DSA2:

mark one node called *the root* as 'in the tree'

repeatedly:

> add the 'first' link that goes to a node not in the tree
> mark newly connected node as 'in the tree'

result = spanning tree

# a careful ordering

algorithm works with any idea of which link/node is first

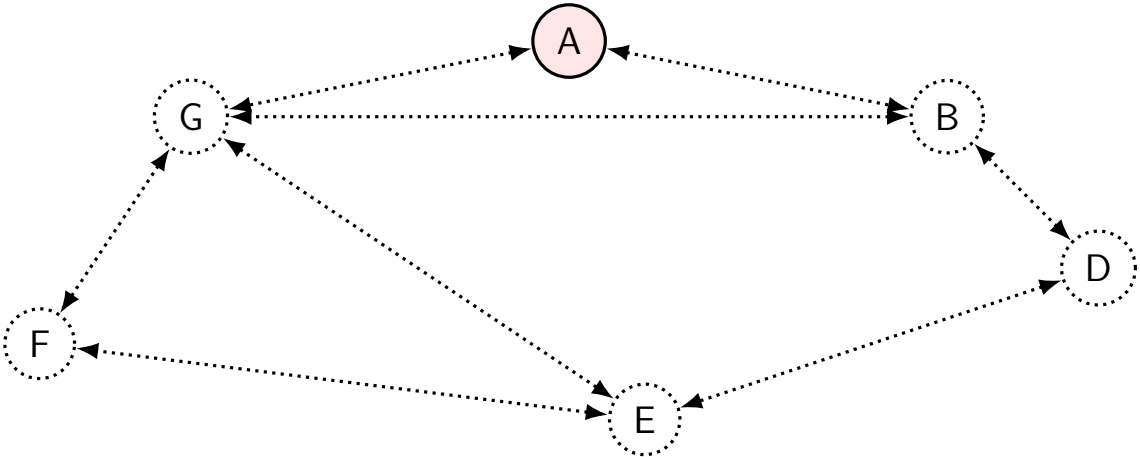
we'll choose a particular ordering (for reasons you'll see later)

first node one with earliest 'name'

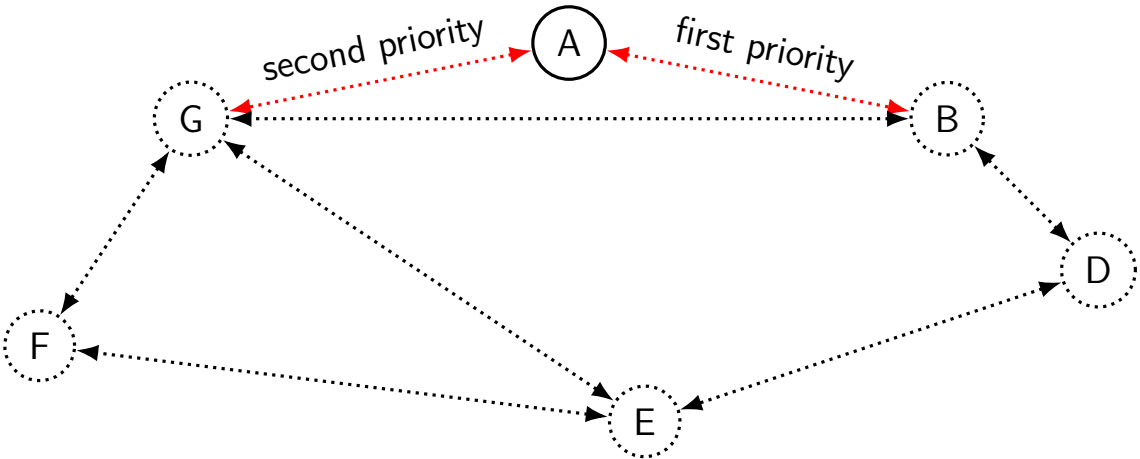
links closer to the root before further links

links from nodes with earlier names before later ones

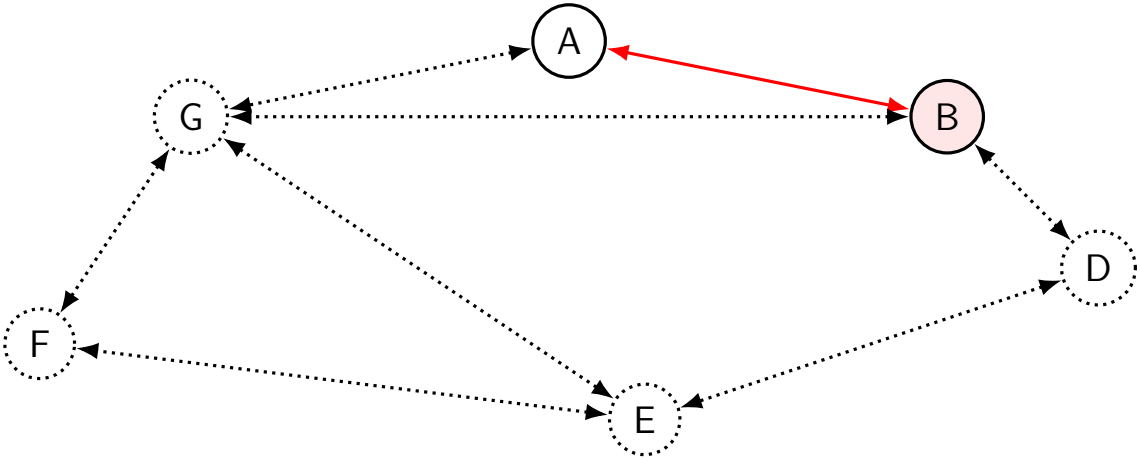
# spanning tree example



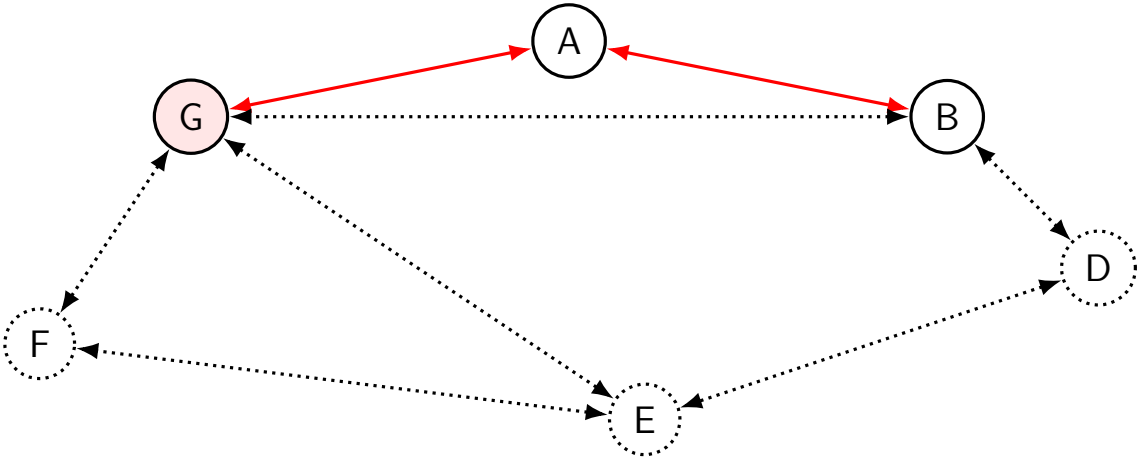
# spanning tree example



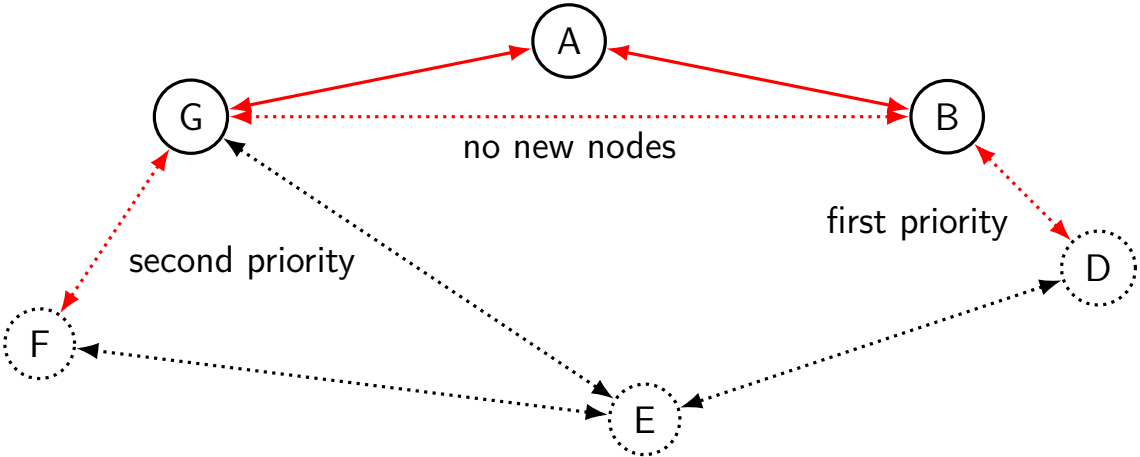
# spanning tree example



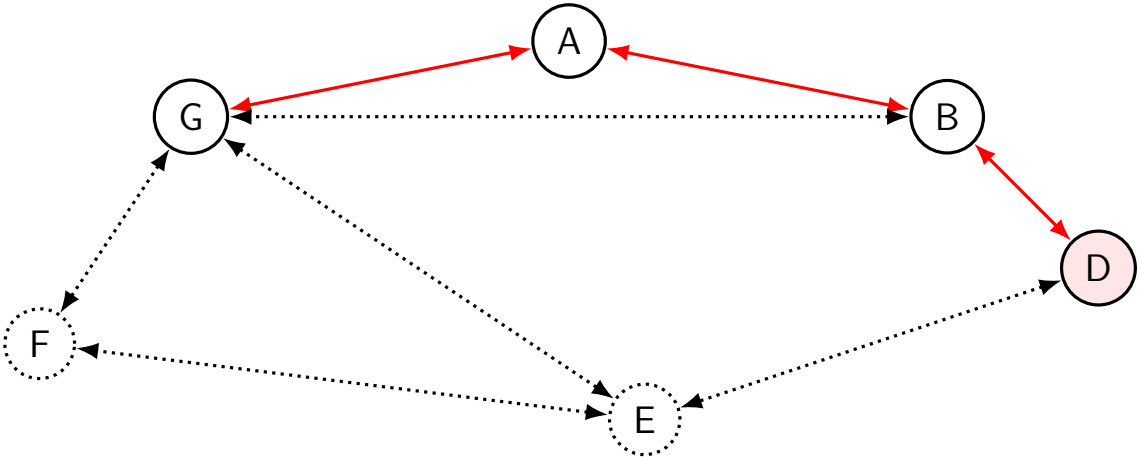
# spanning tree example



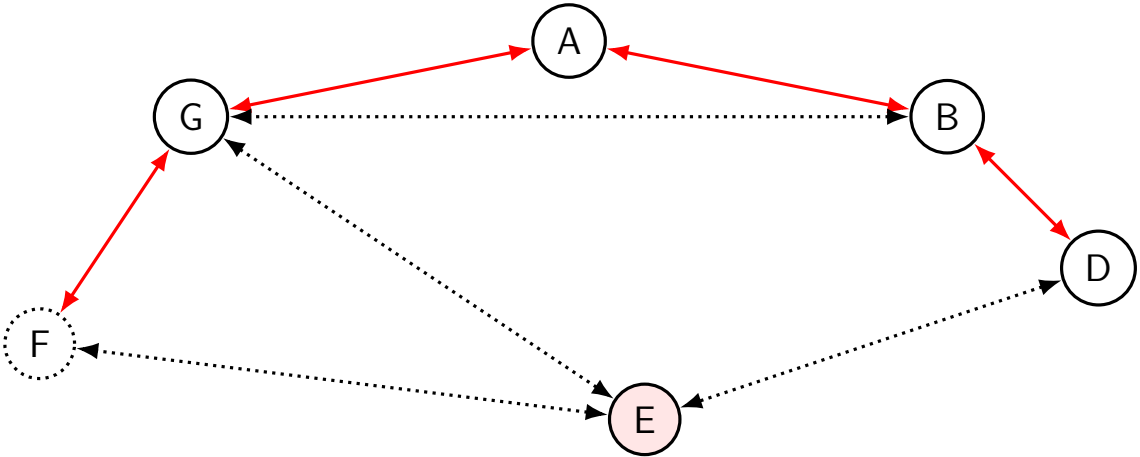
# spanning tree example



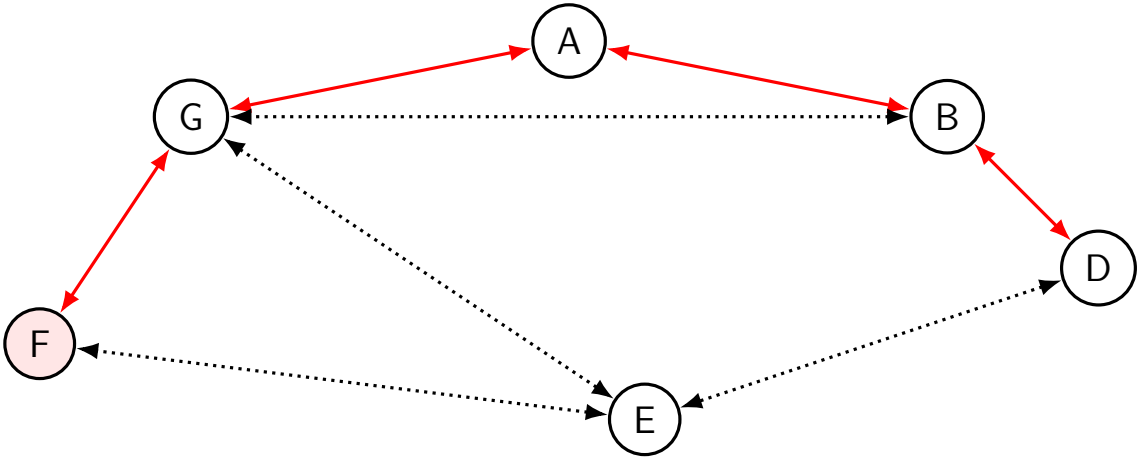
# spanning tree example



# spanning tree example



# spanning tree example



# detecting ‘mistakes’

this method: consistent results every time

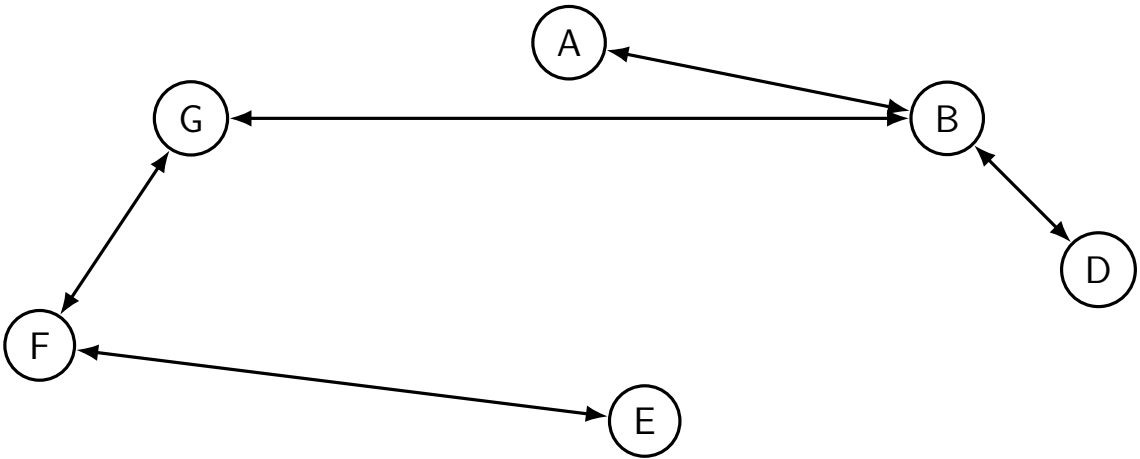
but assumes we start from scratch

we’re going to want a way of doing this dynamically

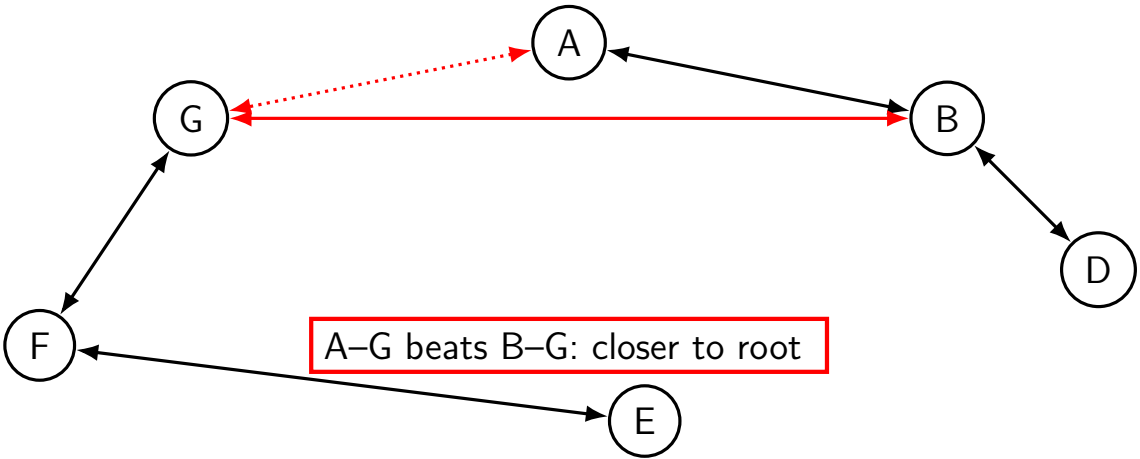
let’s say we find a wrong configuration —

can we fix it?

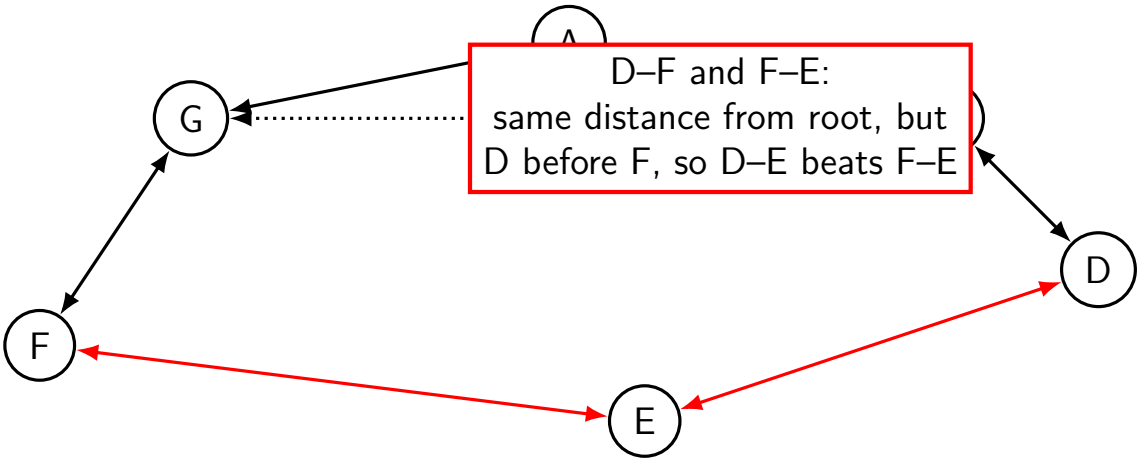
# fixing wrong links



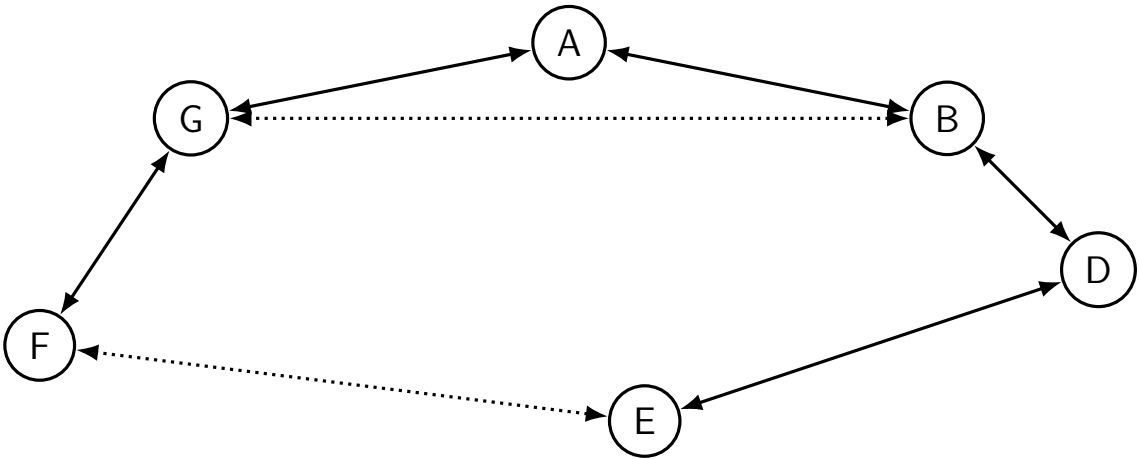
# fixing wrong links



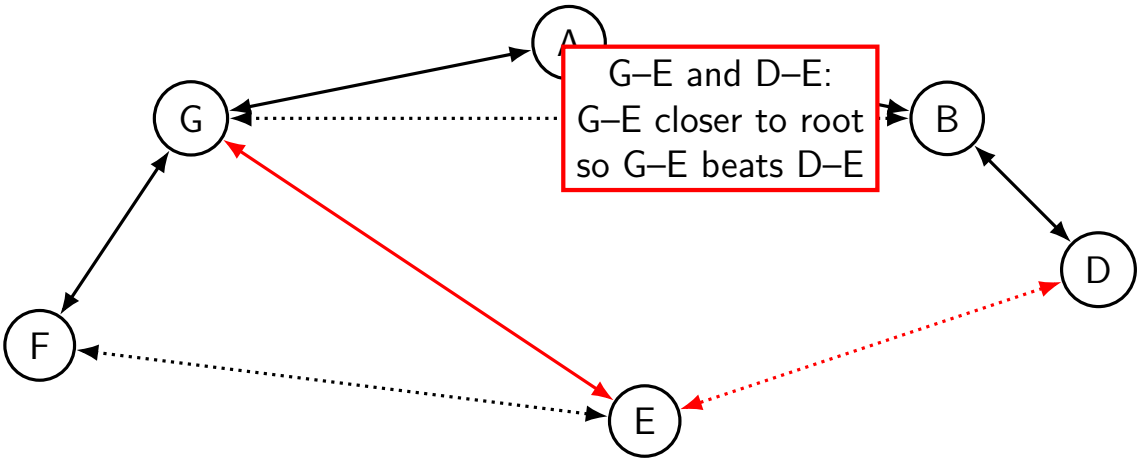
# fixing wrong links



# fixing wrong links



# fixing wrong links



# spanning tree protocol

each node tracks:

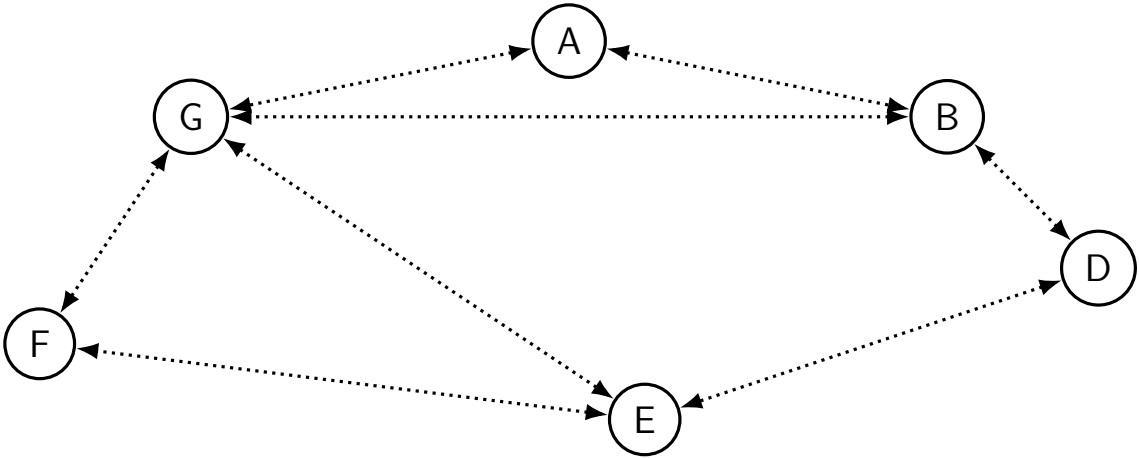
- what it believes is root of tree
- its link toward root of tree
- its distance to root of tree
- which other nodes think it's closer to root of tree

periodically sends information to neighbors

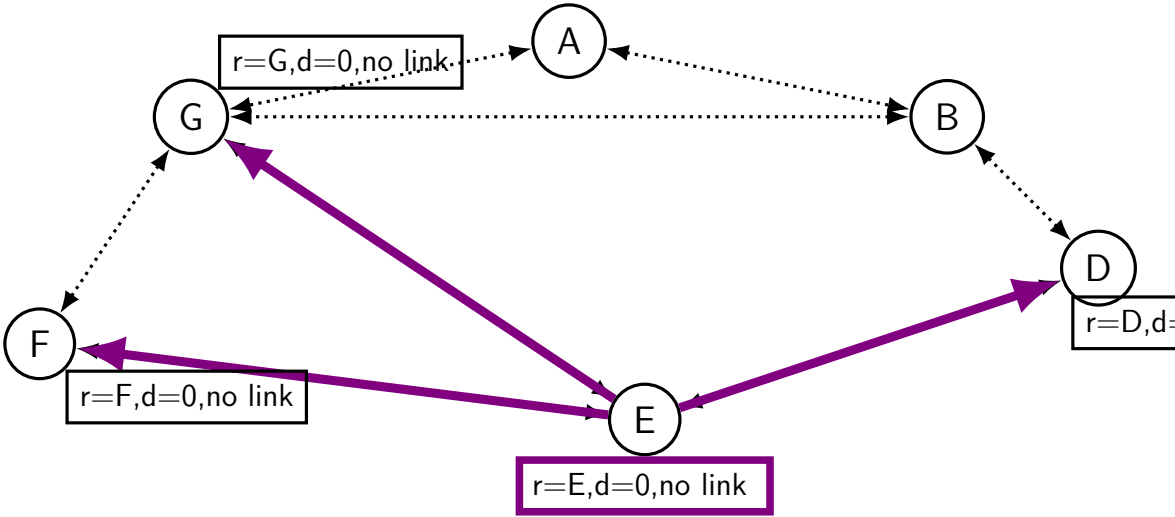
when receiving information, update:

- root to lower ID number (if possible)
- link to lower-distance link (if possible)
- link to lower-ID, same-distance link (if possible)
- which other nodes think it is closer

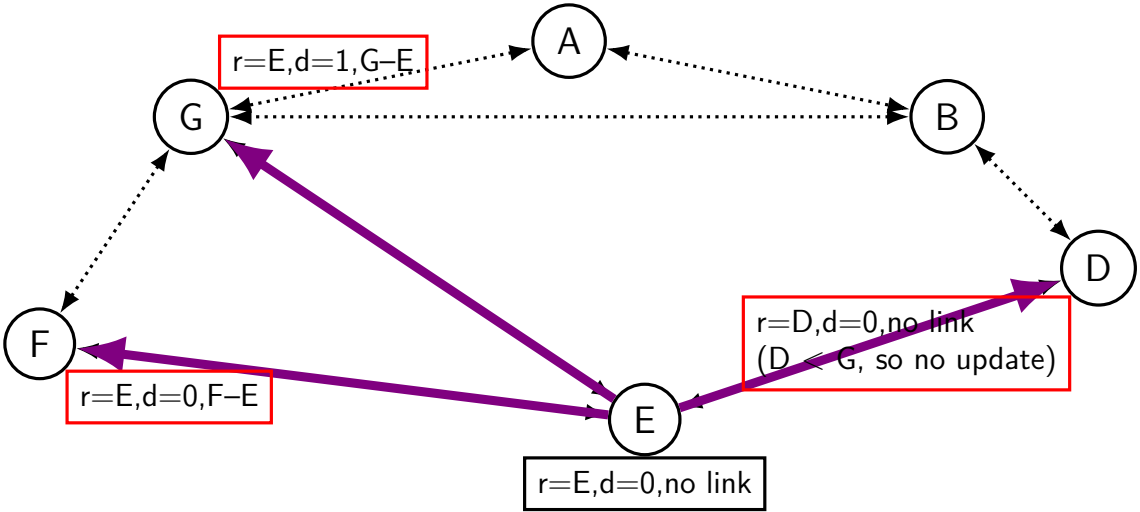
# some example updates



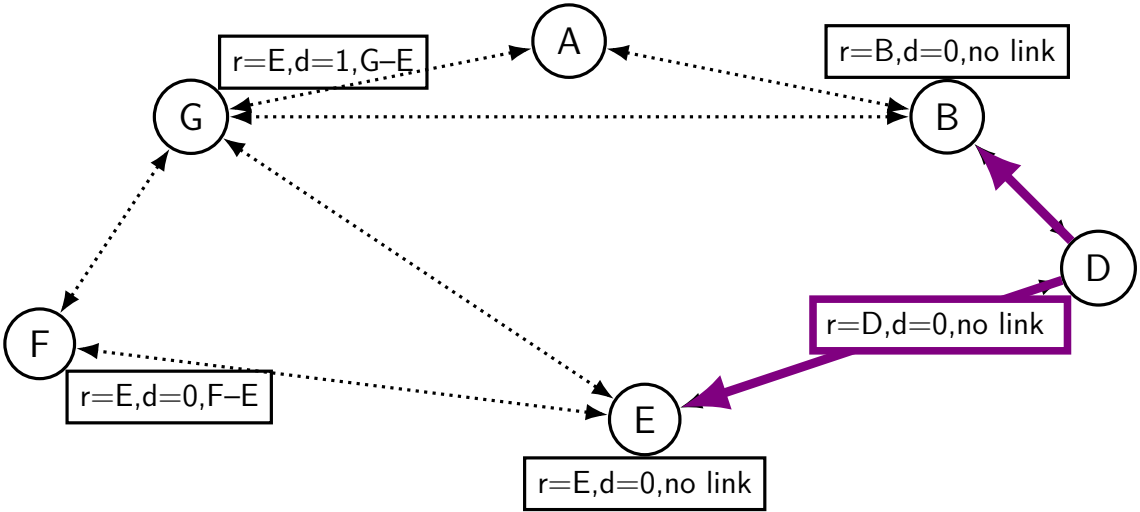
# some example updates



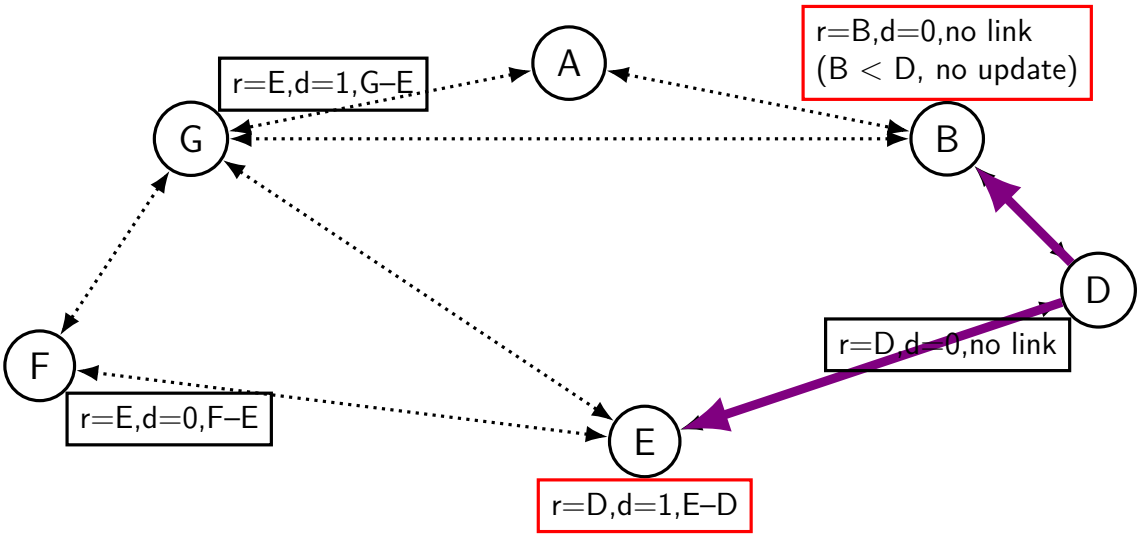
# some example updates



# some example updates



# some example updates



# spanning trees in practice (1)

commonly used on Ethernet for switches

links not in spanning tree are 'blocked'

- not used for normal traffic
- assumption: would cause loop $\rightarrow$ infinite packets

delay before activating port

- avoid temporary routing loops while figuring out tree

periodically send updates to all neighbors

- order of seconds

# spanning tree in practice (2)

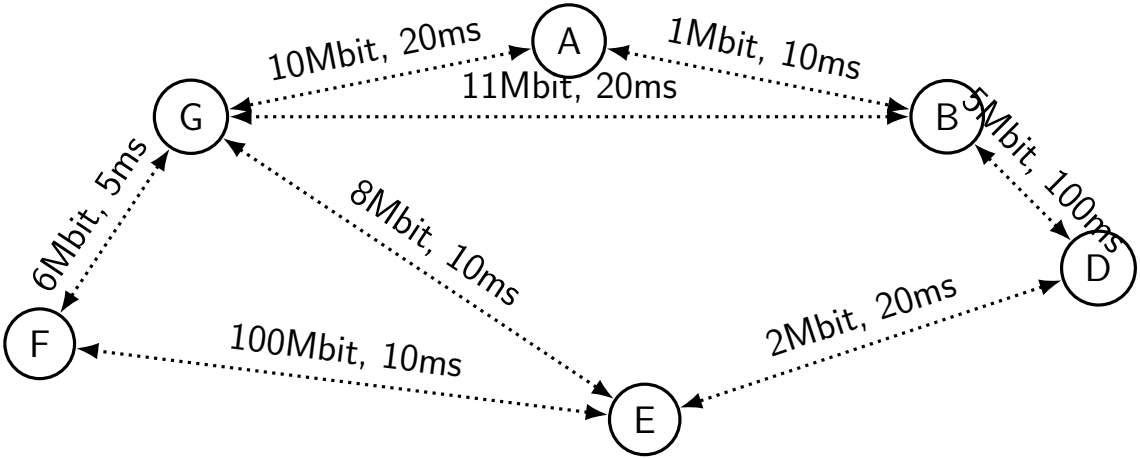
real protocol supports variable ‘cost’ for links

- so ‘distance to root’ might be lower for faster links

modern variant (Rapid Spanning Tree Protocol) selects “backup” port to root

- goal: faster switchover on failure

# exercise: best routes?




A to B? B to E? F to G?

# backup slides